The Systems Behind Modern AI

Why and how does hardware designed for 3D games power modern AI

Sam Ly

Cal Poly Pomona

Spring 2026

Table of Contents

How does Matrix Multiplication map to Neural Networks?

Why GPUs?

Optimizing **Ge**neral Matrix Multiplication (GEMM)

lol

Table of Contents

How does Matrix Multiplication map to Neural Networks?

Why GPUs?

Optimizing **Ge**neral Matrix Multiplication (GEMM)

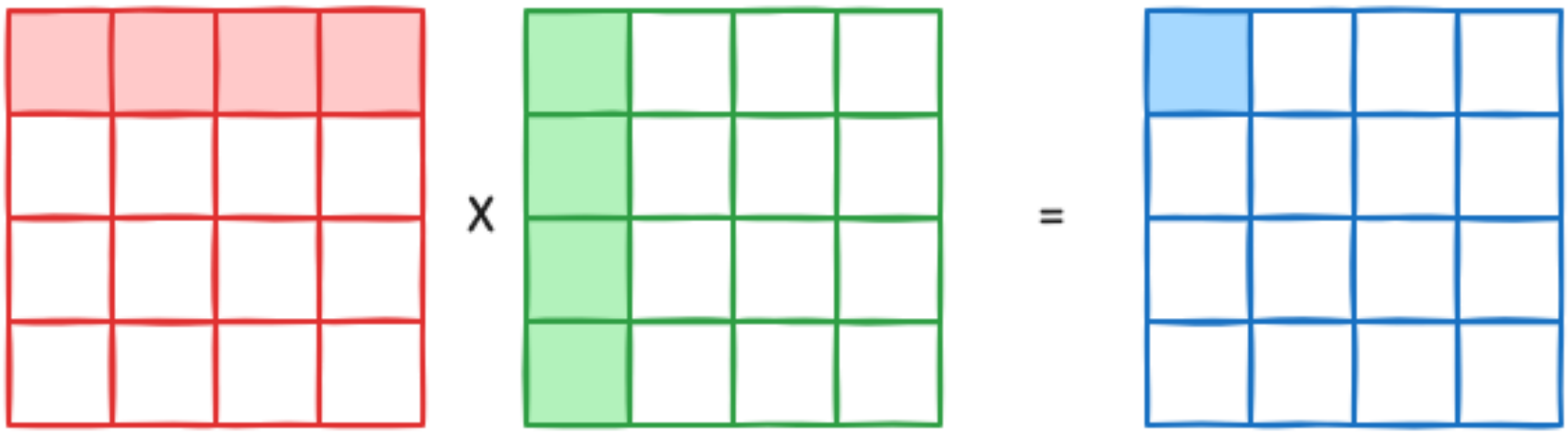
History of GPUs

- 1999: GeForce 256
- 2004: Brook Programming, more complex "rendering" pipelines, GP-GPU
- 2006: GeForce 8800, Compute Unified Device Architecture (CUDA)
- 2008: Fermi, General Purpose Scientific Compute becomes first class, L1/L2 caches
- 2010-2016: incremental improvements to power efficiency, programming models, and multi-GPU systems.
- 2017: Volta, AI-specialization: tensor cores designed around deep learning primitives
- 2022: Hopper, Transformer Engine, further specializing into AI
- 2026: Since 2022, we have only really seen incremental improvements.

Parallelism

Graphics rendering is a naturally parallel task. Each pixel does the same work.
CPUs can't keep up, so we trade logical capability for **throughput**.
Each *thread* on the GPU can only do simple things, but we have many of them. GPU's can't do some of the more algorithmically complex tasks, but can do a a large amount of simple tasks much faster.
That is, a CPU has few threads that are very capable. A GPU has many dumb threads.

Matrix Multiplication is Parallel



The operations are very simple, but the number of operations grow on the order of $\mathcal{O}(n^3)$.

CPU implementation

Table of Contents

How does Matrix Multiplication map to Neural Networks?

Why GPUs?

Optimizing **Ge**neral Matrix Multiplication (GEMM)

Naive

Arithmetic Intensity, throughput, and the Roofline Model